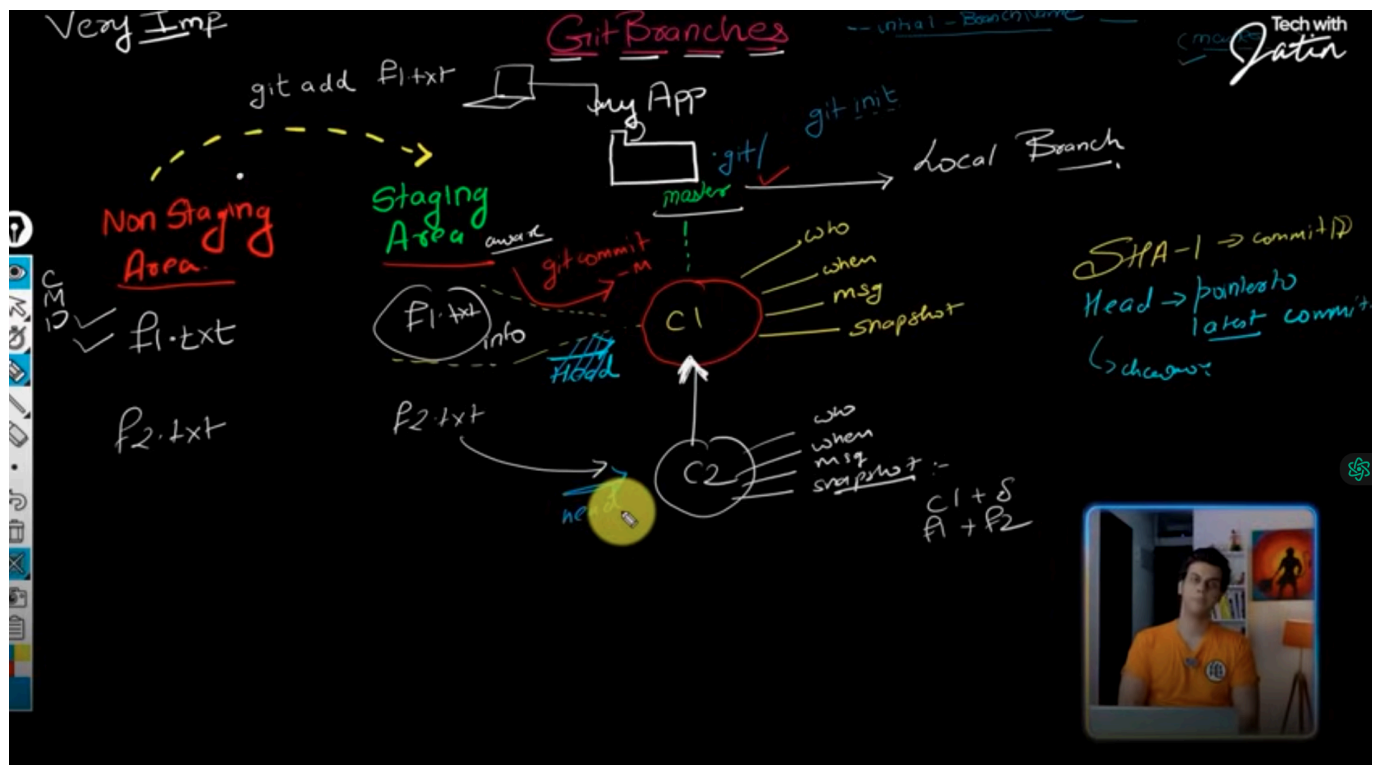


What is a Branch?

A branch is a separate workspace in Git used to develop features independently.

Branch

- Branches are always connected with commits.
- A branch is created in the local repository.
- **HEAD** is a pointer to the latest commit.
- Developers should work on their own branch, not directly on the main branch.
- If two branches point to the same commit, it means both have the same content.



Git Branch Commands

Show All Branches

- This command shows all branches in the repository.

```
git branch
```

Create a New Branch

- This command creates a new branch named **taha**.

```
git branch taha
```

Same Commit Pointer

- If two branches point to the same commit, it means both have the same content

```
TAHA@DESKTOP-HUSIIRO MINGW64 ~/Desktop/git and github practise/one (main)
$ git log
commit 9a2d704c784b39cd17cc822e9229c933495f7c98 (HEAD -> main, taha)
Author: taha <muhammادتaha00224443@gmail.com>
Date: Sat May 9 18:14:08 2026 +0500

    Revert "three:"
```

How to Switch Branch

```
git checkout taha
```

```
TAHA@DESKTOP-HUSIIRO MINGW64 ~/Desktop/git and github practise/one (main)
$ git checkout taha
Switched to branch 'taha'

TAHA@DESKTOP-HUSIIRO MINGW64 ~/Desktop/git and github practise/one (taha)
$
```

Commit in Another Branch

- Changes and commits will only save in the current branch.
- The main branch will not get those changes until merge happens.

```
vi f2
git add f2
git commit -m "taha branch first"
```

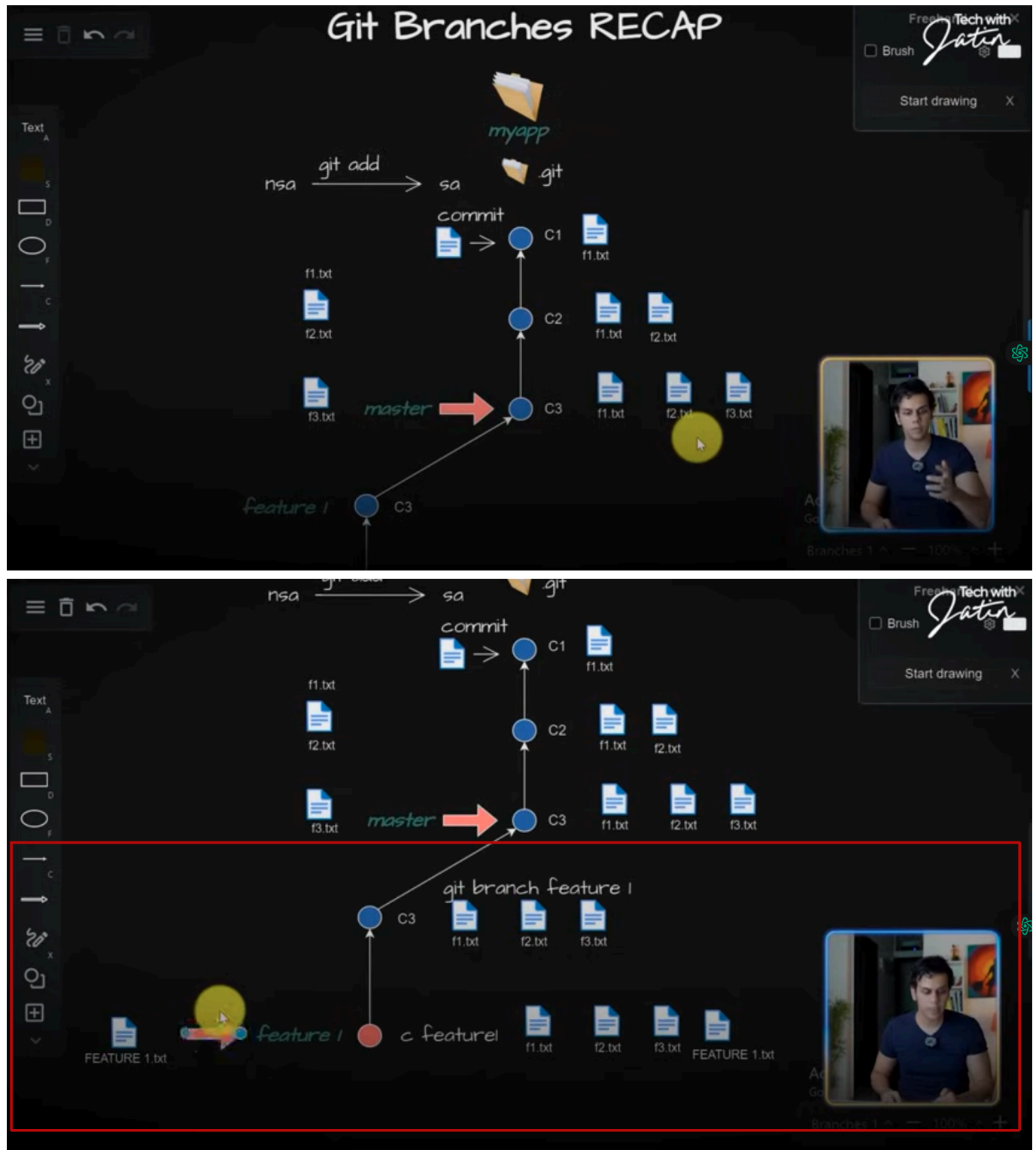
What is Isolation?

- Branch isolation means changes in one branch do not affect another branch.
- You can create a branch from any existing branch.

How to Delete a Branch

```
git branch -d taha
```

Git Branch Workflow



- Before the first commit, the **main** branch name may appear, but **git branch** does not show any branch.
- Git officially creates the branch after the first commit.

```
mkdir new-project
```

```
git init
```

```
touch index.html
```

```
git add .
```

```
git commit -m "first"
```

```
git status
```

```
vi index.html
```

```
<!DOCTYPE html>
<html>
<head><title>Home</title></head>
<body>
<nav>
<a href="#">Home</a>
<a href="#">About</a>
<a href="#">Contact</a>
</nav>
<h1>Welcome to My Website</h1>
</body>
</html>
```

- After updating the file, Git shows the file as modified.

```
git add .
```

```
git commit -m "edit index.html"
```

Create and Switch Branches

```
git branch taha
```

```
git log
```

- (HEAD -> main, taha) means both branches currently contain the same commits and content.

```
git checkout taha
```

```
git switch main
```

- `git switch` is also used to change branches.

Branch Update Concept

- After switching to the `taha` branch, update the `index.html` file.
- The new updated code exists only in the `taha` branch, not in `main`.
- These are all local branches.
- Now the new commit needs to be shared with the `main` branch.
- After task completion, branches can be merged.
- To receive new code, you must switch to the target branch first.

```
git merge taha
```

- `taha` branch is one step ahead of `main`.
- `main` branch is one step behind `taha`.

```
</html>
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/two (main)
$ git merge taha
Updating 17d4232..a649cb0
Fast-forward
 index.html | 3 +++
 1 file changed, 3 insertions(+)

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/two (main)
$ cat index.html
<!DOCTYPE html>
<html>
<head><title>Home</title></head>
<body>
<nav>
<a href="#">Home</a>
<a href="#">About</a>
<a href="#">Contact</a>
<h1>taha</h1>
<h1>fatima</h1>
<h1>kashaf</h1>
</nav>
<h1>Welcome to My Website</h1>
</body>
</html>

TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/two (main)
$
```

Simple Git Branch Merging

```
git branch fatima
```

```
git checkout fatima
```

```
touch fatima.txt
```

```
git add .
```

```
git commit -m "first commit from fatima"
```

- Now switch back to the `main` branch.

```
git checkout main
```

```
git merge fatima
```

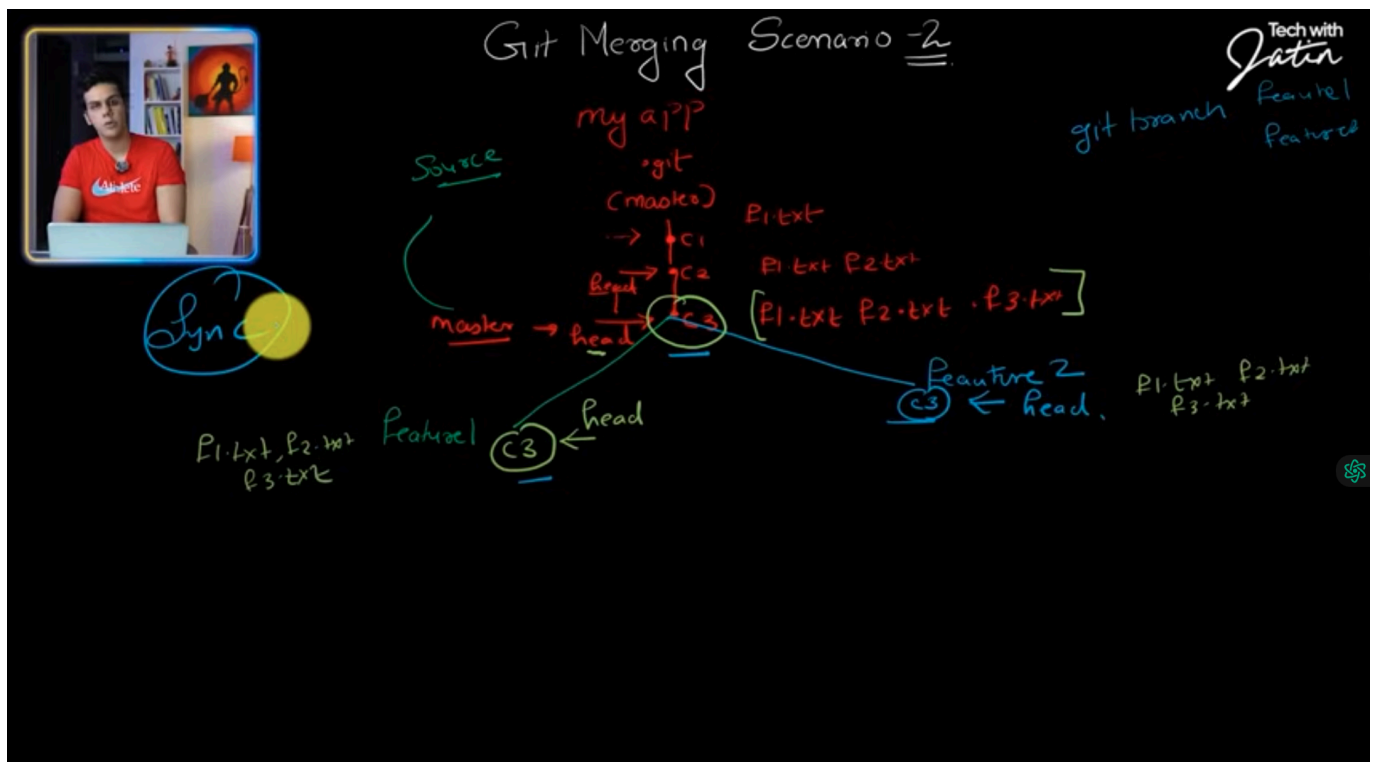
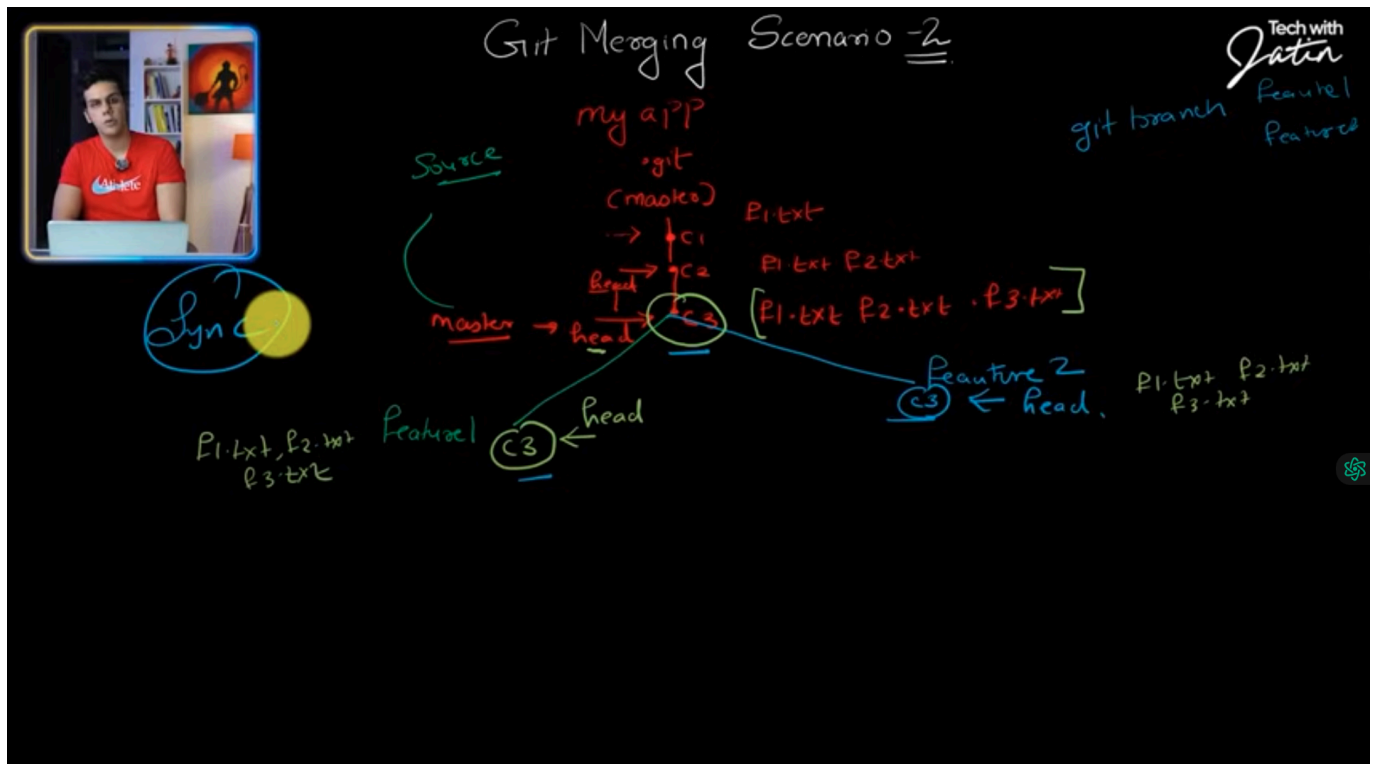
- This command merges the **fatima** branch into the **main** branch.
- If the **taha** branch also needs the same file or updates, then switch to the **taha** branch.

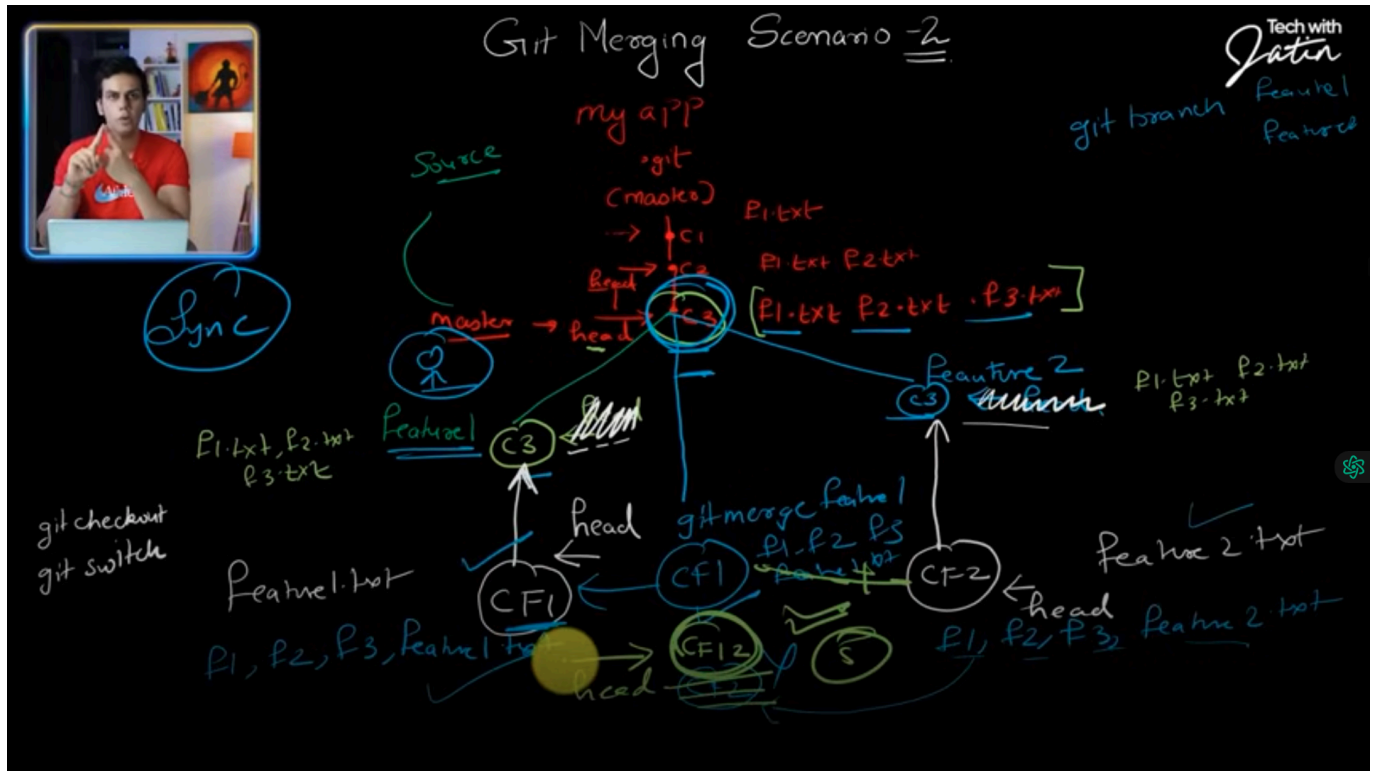
```
git switch taha
```

```
git merge fatima
```

- This command merges the **fatima** branch into the **taha** branch.
- This is called simple merging.
- Next, multiple branch merging can also be performed in Git.

Multiple Branch Merging





```
git init
```

```
touch file1 file2 file3
```

```
git add .
```

```
git status
```

```
git commit -m "created file 1, 2, 3"
```

```
git branch taha
```

```
git branch fatima
```

```
git log
```

```
$ git log
commit b1cd9fd35c685af73b8e893fb08aa36a60a7fc82 (HEAD -> taha, main, fatima)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sun May 10 01:17:25 2026 +0500

    first commit from fatima
```

- This means all branches are currently synchronized and contain the same commits.

Work on taha Branch

```
git switch taha
```

```
touch taha.txt
```

```
git add .
```

```
git commit -m "taha.txt by taha"
```

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/two (taha)
$ git log
commit d63f0e123f7469de55b63e19d901c6e0798ecb42 (HEAD -> taha)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sun May 10 09:48:50 2026 +0500

    taha.txt by taha

commit b1cd9fd35c685af73b8e893fb08aa36a60a7fc82 (main, fatima)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sun May 10 01:17:25 2026 +0500

    first commit from fatima

commit a649cb0ded6019e076fd59d2ac0acd01f62de467
Author: taha <muhammادتaha00224443@GMAIL.COM>
```

- This means `main` and `fatima` are synchronized.
- `taha` is one step ahead of both branches.

Work on **fatima** Branch

```
git switch fatima
```

```
touch fatima.txt
```

```
git add .
```

```
git commit -m "fatima.txt by fatima"
```

Merge Branches into **main**

```
git switch main
```

```
git merge taha
```

- This merge happens directly without any issue because **main** has no extra commits.

Merge **fatima** into **main**

```
git merge fatima
```

- Now Git creates a new merge commit because both branches contain different commits.
- Vim editor opens to write the merge commit message.
- Git automatically writes a message like **Merge branch 'fatima' into main**.

```
TAHA@DESKTOP-HUSIIRQ MINGW64 ~/Desktop/git and github practise/two (main)
$ git log
commit 38853ad00be3550bab24779fe358e8edbd92ba9d (HEAD -> main)
Merge: d63f0e1 04e5d0f
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sun May 10 10:31:04 2026 +0500

    Merge branch 'fatima' into main branch

commit 04e5d0f0fc4e35ac6624eca5145163e4174c9465 (fatima)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sun May 10 10:25:58 2026 +0500

    fati by fatima

commit d63f0e123f7469de55b63e19d901c6e0798ecb42 (taha)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sun May 10 09:48:50 2026 +0500

    taha.txt by taha

commit b1cd9fd35c685af73b8e893fb08aa36a60a7fc82
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Sun May 10 01:17:25 2026 +0500

    first commit by fatima
```

Git Merge Conflict

- A merge conflict occurs when two developers edit the same file differently.
 - Git cannot decide which changes should be kept automatically.
 - In this case, the conflict must be resolved manually.
-

Start Conflict Practice

```
git checkout taha
```

```
git merge main
```

- (HEAD -> fatima, taha, main) means all branches are synchronized.
-

```
git checkout fatima
```

```
git merge main
```

- (HEAD -> fatima, taha, main) means all three branches contain the same commits.

Edit File in fatima Branch

```
vi file1
```

- Add content:

```
this content from fatima
```

```
git add file1
```

```
git commit -m "file1 edit"
```

Edit Same File in taha Branch

```
vi file1
```

- Add content:

```
this content from taha
```

```
git add file1
```

```
git commit -m "file1 edit"
```

- Now the same file has been edited differently by two developers.

- The file content is different in both branches.
- If you merge now, a conflict will occur.

Merge Conflict

- Git merge fails because of conflicting changes.
- Git asks to fix the conflict first, then complete the merge.
- Open the file in a text editor and remove the extra conflict lines added by Git.
- Keep only the correct content.
- After fixing the file, it moves back to the non-staging area.

```
git add .
```

```
git commit -m "resolve merge conflict"
```

- This command creates a new commit after resolving the conflict.

Synchronize All Branches

```
git switch fatima
```

```
git merge taha
```

```
git switch main
```

```
git merge taha
```

- (HEAD -> main, taha, fatima) means all branches are now synchronized.
-
-
-
-

Git Cherry-Pick

- In this lecture, GitHub, GitBucket, and GitLab will be discussed.

```
git switch taha
```

```
touch sample
```

- Add content inside the file:

```
this is sample content from taha
```

```
git add .
```

```
git commit -m "sample taha"
```

- Now the `taha` branch commit needs to be added into the `main` branch without using the merge command.

```
git cherry-pick commit-id
```

- `git cherry-pick` copies a specific commit from one branch to another branch.
- Git creates the same commit copy in the current branch.

```
# Git Cherry-Pick
- specific commit ko ek branch sa dusri branch ma copy krna

# commit dekhna
git log --oneline

# branch change
git checkout main

# cherry-pick
git cherry-pick commitID

# multiple commits
```

```
git cherry-pick id1 id2

# conflict continue
git add .
git cherry-pick --continue

# cancel cherry-pick
git cherry-pick --abort

# use
- sirf important commit copy krna
- full merge avoid krna
```

Git Multi-line Commit Message

```
mkdir git-project
```

```
cd git-project
```

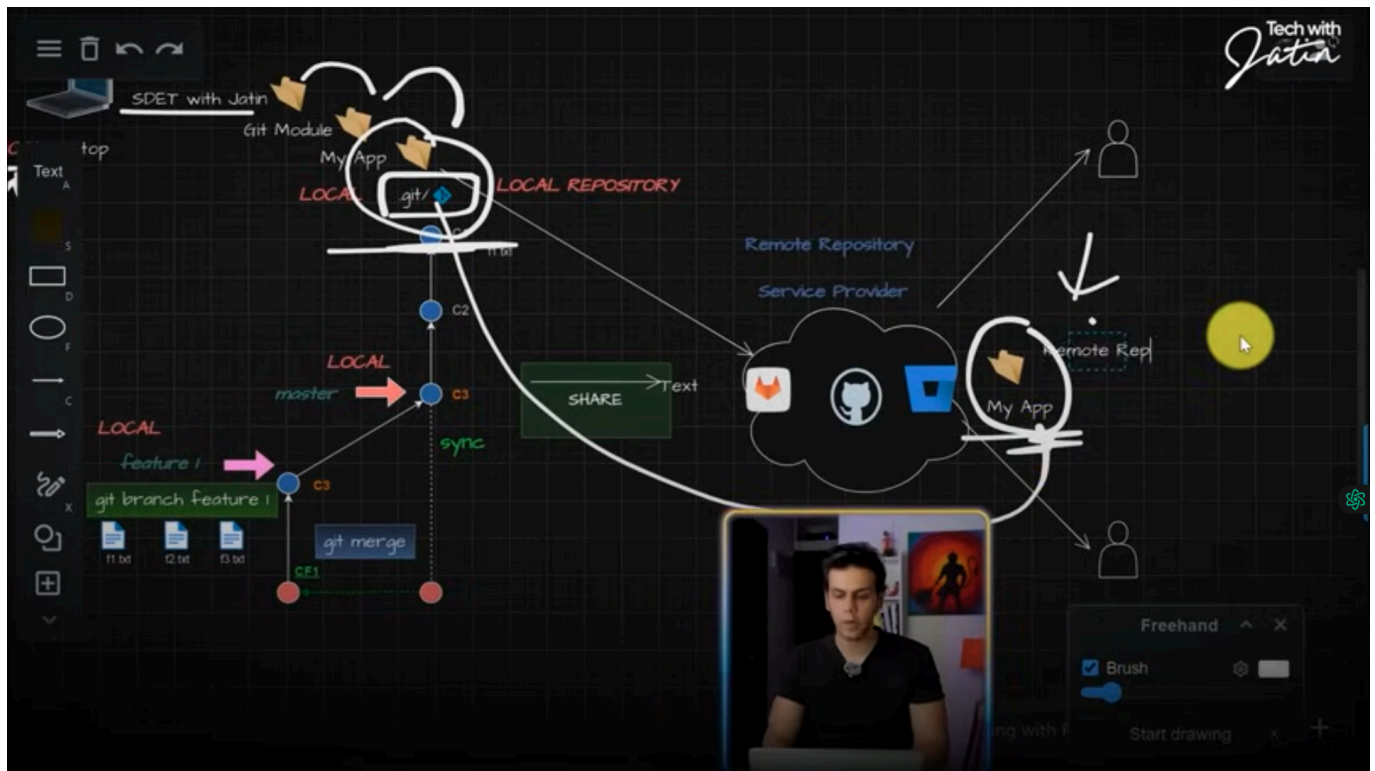
```
mkdir E-commerce
```

```
touch login.html
```

```
git commit -m "subject" -m "body means full commit explanation"
```

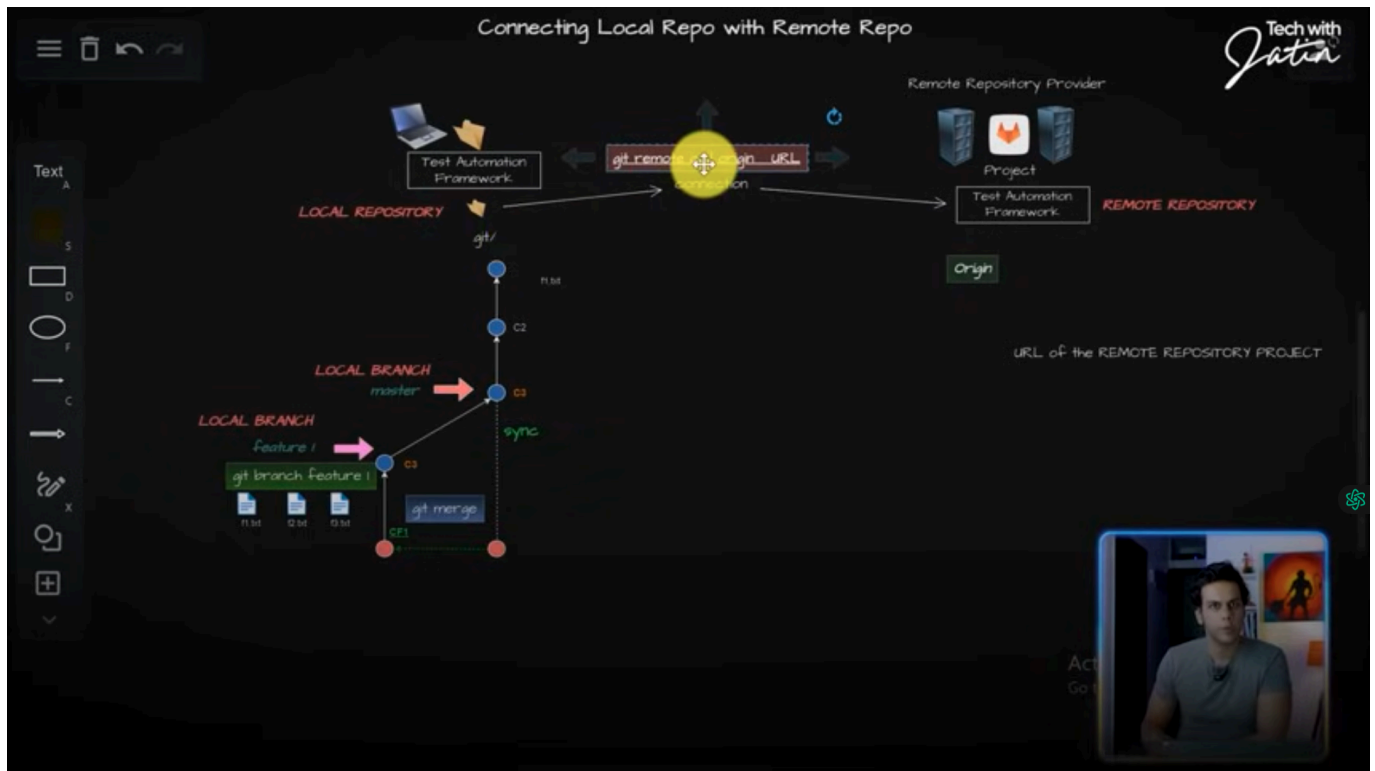
- The first `-m` is used for the commit subject/title.
- The second `-m` is used for the detailed commit description.

-
-
-
- When we create a Git repository on a laptop or computer, it is called a local repository.
 - All branches created inside it are also local branches.



- A remote repository is used to share projects with other people.
- GitLab account can be created using an email address.
- A group can also be created in GitLab for practice projects.

How to Connect Local Repository with Remote Repository



pty

- To connect a local repository with a remote repository, use:

```
git remote add origin https://gitlab.com/practise-project/gitlab-practise.git
```

- `origin` is the default name of the remote repository.
- This command links the local repository with GitLab.

How to Check Remote Repository

```
git remote
```

- This command shows connected remote repository names.

How to Push Changes to Remote Repository

```
git push -u origin main
```

- `git push` uploads the local repository changes to the remote repository.
- `-u` sets the upstream branch connection.

- If the branch does not exist on the remote repository, Git creates it automatically.
- `origin` is the remote repository name.
- `main` is the branch name being pushed.

Git Remote Branch Information

```
commit b64eff4a4af2e1e245397a860896d01640d1f289
Author: taha <muhammادتaha00224443@gmail.com>
Date: Sat May 9 08:50:38 2026 +0500
...skipping...
commit e95783c36df2f34bb50dcb47f821fcb2217fbcd1 (HEAD -> taha, origin/taha)
Author: taha <muhammادتaha00224443@GMAIL.COM>
Date: Mon May 11 09:14:55 2026 +0500

    taha-file

commit ca41f0bfde3c4fa8b3d7015f860aa1fcb7465bb3 (origin/main, main)
Author: taha <muhammادتaha00224443@gmail.com>
Date: Sat May 9 09:46:50 2026 +0500

    dummy

commit b5206b8e025650642652811e0b58acch4020d2eb
```

- `(HEAD -> taha, origin/taha)` means the local `taha` branch and remote `origin/taha` branch are synchronized.
- Both branches contain the same commits and changes.

```
git branch
```

- `git branch` shows local branches only.

```
git branch -r
```

- `git branch -r` shows remote branches.

Git Clone

- `git clone` is used to copy an existing remote repository into the local system.
- This command is commonly used when joining a new company or working on an existing project.

```
git clone url
```

- `git clone` downloads the complete project with all branches and commit history.

- Developers should avoid working directly on the `main` or `master` branch.
 - Always create a separate branch before starting work.
-
-
-

Git Clone, Fetch, and Pull

- `git clone` is used when you do not already have the project on your local system.
 - It downloads the full repository from the remote server to your computer.
-
- If you want to add or update changes made by other developers, you use `git fetch` or `git pull`.
 - These commands are used to download updates from a remote repository.
-

Git Fetch

```
git fetch
```

- This command downloads changes from the remote repository.
 - It does not update your local branch automatically.
 - It only brings updates to your local repository.
-

```
git checkout origin/main
```

- This command is used to view the remote branch state.
-

```
git switch main
```

```
git merge origin/main
```

- This updates your local `main` branch with remote changes.
-

Git Pull

```
git pull
```

- `git pull` combines both `fetch` and `merge`

Git Fetch

- `git fetch` downloads new changes from the remote repository into the local repository.
- It does not automatically merge changes, so you can first review what has changed.
- This helps avoid confusion and conflicts because you can see updates before applying them.

- If a remote branch is deleted and you don't need it locally anymore, use:

```
git fetch --prune
```

- `--prune` removes deleted remote branches from your local tracking list.

- `git fetch` downloads updates from all remote branches.
- It is usually not required to use advanced options frequently.

```
git fetch --all
```

- This command fetches updates from all remote repositories (e.g., GitHub, GitLab, etc.).

Git Pull

- `git pull` is a combination of `git fetch` and `git merge`.
- It directly downloads and merges remote changes into your current branch.
- It is used when you want to quickly update your branch with remote changes.

```
git pull origin branch-name
```

Git Stash

- `git stash` is used when you want to temporarily save your uncommitted changes.
- It allows you to switch branches without committing incomplete work.

```
git stash
```

- This saves your changes in a temporary storage area.
-

```
git stash list
```

- This shows all saved stash entries.
-

- When you return to your work, you can apply saved changes again:

```
git stash apply stash-id
```

- To remove all stash entries:

```
git stash clear
```

.gitignore File

- `.gitignore` is a very important file in Git.
- It is used to ignore specific files or folders from being tracked by Git.
- Files listed in `.gitignore` will not be added to Git repository.

```
vi .gitignore
```

- You can define files like logs, environment files, or sensitive data inside it.